

Библиотеки използващи Nix

Универсални конфигурации с Nix

Павел Атанасов Камен Младенов

29.05.2025

Преговор

- Решавахме задачи с `flake-ове`

Градене над основите на Nix

- До сега разглеждахме и разработвахме неща, използвайки `builtins`, `nixpkgs` и `V3` команди
- Има популярни Nix библиотеки, които се срещат в голямо множество от конфигурации
- Има проекти където Nix се използва като инструмент за DevOps процеси

Сечение 1

flake parts

Какво е flake-parts?

- **flake-parts** е библиотека, която ни дава начин да използваме (вече добре познатата ни) NixOS модулна система в контекста на flake-ове
- Сега ще разгледаме как можем да използваме тази библиотека, че да опростим големите си flake-ове

Базов Пример

```
{
  inputs = {
    nixpkgs = {
      url = "github:NixOS/nixpkgs/nixos-unstable";
    };
    flake-parts = {
      url = "github:hercules-ci/flake-parts";
    };
  };

  # outputs = ...
}
```

```
{  
  # inputs = ...  
  
  outputs = inputs: inputs.flake-parts.lib.mkFlake { inherit inputs; } (  
    { config, inputs, self, ... }: {  
      systems = [ "x86_64-linux" "aarch64-linux" ];  
      imports = [ ];  
  
      flake = { top = "level"; };  
      perSystem = { config, inputs', self', pkgs, ... }: {  
        packages.hello = pkgs.hello;  
  
        devShells.default = pkgs.mkShell { buildInputs = [ pkgs.hello ]; };  
      };  
    });  
}
```

До какво се оценява това?

```
{  
  # ...  
  devShells = {  
    aarch64-linux.default = «derivation /nix/store/...-nix-shell.drv»;  
    x86_64-linux.default = «derivation /nix/store/...-nix-shell.drv»;  
  };  
  # ...  
  packages = {  
    aarch64-linux.hello = «derivation /nix/store/...-hello-2.12.1.drv»;  
    x86_64-linux.hello = «derivation /nix/store/...-hello-2.12.1.drv»;  
  };  
  # ...  
  top = "level";  
}
```


Какво се случи тук?

- Стойността на `outputs` се определя изцяло от резултата на извикването на `mkFlake` от `flake-parts`, на която ѝ подаваме `inputs`
- Оттам то автомагически си извлича `nixpkgs`, от което си създава канонична `pkgs` инстанция (може да се конфигурира с `_module.args`, my beloved)

- И финално, ние му подаваме атрибутното множество*, което дефинира няколко атрибута (има и още поддържани), а именно:
 - ▶ `flake` дефинира *toplevel* неща, които директно ще се появят във финалния резултат
 - ▶ `perSystem` дефинира `system`-зависими неща като `packages`, `devShells`, т.н.

mkFlake и аргументите му

- `config` - подобно на NixOS модулите, *финалната* версия на отварата в казана след обединяване на всички модули
- `self` - нещо *като* `config`, ама е как ни изглежда `flake`-а отвън (без `flake` и `perSystem` абстракции)

perSystem и аргументите му

- `config` - като глобалното `config`, ама е само за `perSystem` нещата, за текущия `system`
- `inputs'` - като глобалното `inputs`, но `system`-зависимите неща са *смачкани* (няма `${system}` атрибут, през който да минаваме)
- `self'` - като глобалното `self`, но нещата, които са дошли от `perSystem` са (подобно на `inputs'`) *смачкани*
- `pkgs` - както предполагахме, това е инстанцииран `nixpkgs` с *правилния system*
- ...

Сечение 2

home manager

Какво е home-manager

- Nix изгражда възпроизводими *глобални* и локални среди, но никога не менажираме HOME директорията
- `home-manager` е библиотека, която позволява менажиране на всичко под HOME, използвайки Nix
 - ▶ конфигурационни файлове
 - ▶ програми

Базов пример

Home-manager Nix израз

```
programs.git = {  
  enable = true;  
  userEmail = "joe@example.org";  
  userName = "joe";  
};
```

Резултатен файл ~/.config/git/config

```
[user]  
  email = "joe@example.org"  
  name = "joe"
```

Сравнение с пазене на резервни копия, хранилища, ...

- Описаните съдържания в HOME са напълно възпроизводими, включително програмите които се конфигурират
- По-ефикасно, защото Nix кодът ще заема по-малко място от множества копия/commit-и/... и ще позволява по-бързо избиране на версии
- Композируемо с цялата останала екосистема на Nix: менажиране на множества HOME директории спрямо потребител и/или машина, генериране на конфигурационни файлове спрямо глобални конфигурации в системата, ...

Сечение 3

DevOps и terranix

Какво е DevOps

- От началото, софтуерната индустрия се е разделяла на две части:
 - ① Development - разработката на софтуер (програмирането)
 - ② Operations - менажиране, поддръжка, мониторинг на софтуер и съответната инфраструктура
- Наемани са напълно различни екипи за всяка част, работещи в изолация (*и конфликт*)

- През 2007 се заражда идеята за DevOps: обединение на development и operations чрез **автоматизация**
- Описване на инфраструктура (машини, програмите върху тях, конфигурациите в тях, ...) чрез код
- **Това е основната цел на Nix**
- [Integrating Software Construction and Software Deployment](#) (2003)
Първия whitepaper написан за Nix

nixos-rebuild

- Използвахме го в [упражнението върху flake-ове](#), за да компилираме виртуални машини спрямо NixOS конфигурации (`nixos-rebuild build-vm`)
- Освен това можем да изпълним и `nixos-rebuild switch`. Тази команда:
 - ❶ Компилира системната конфигурация
 - ❷ Добавя я в root-овете (т.е. да не се изтрива по подразбиране)
 - ❸ “Активира” я (прилага всички възможни промени без да рестартира компютъра)
 - ❹ Добавя референция в bootloader-a

- Към `nixos-rebuild switch` можем да добавим и аргументите `--target-host` и `--build-host`
- Първия ни задава към коя машина (по подаден `[USER]@[ADDRESS]`) да изпълним действието
- Втория посочва коя машина ще извърши компилацията на конфигурацията
- Ето така ще реализираме “deployment”

Популярни библиотеки

- [nixops4](#) Автоматизиран deployment на NixOS машини в мрежа или облак
- [terranix](#) Дефиниране на [Terraform](#) конфигурации (IaC) през Nix
- [deploy-rs](#) Автоматизиран deployment на Nix flake-ове
- [colmena](#) Модерен автоматизиран deployment на NixOS машини

- **Terraform** е популярен софтуер, предоставящ IaC (infrastructure as code)
- **terranix** е Nix библиотека, която ни позволява да опишем **Terraform** конфигурации използвайки единствено Nix код

Базов terranix пример

flake.nix

```
{
  inputs = {
    terranix.url = "github:terranix/terranix";
    terranix.inputs.nixpkgs.follows = "nixpkgs";
  };
  outputs = { terranix, ... }: {
    defaultPackage.${system} = terranix.lib.terranixConfiguration {
      system = "x86_64-linux";
      modules = [ ./config.nix ];
    };
  };
}
```



```
{ lib, ... }: {  
  variable.hcloud_token.sensitive = true; # Hide API token from VCS  
  provider.hcloud.token = "\${var.hcloud_token}"; # Cloud provider  
  
  resource.hcloud_server.my_server = {  
    name      = "myserver.example.org";  
    image     = "debian-13"; server_type = "cx23"; location = "nbg1";  
    ssh_keys  = [ "\${hcloud_ssh_key.my_key.id}" ];  
    public_net = { ipv4_enabled = true; ipv6_enabled = true; };  
  };  
  resource.hcloud_ssh_key.my_key = {  
    name      = "my-ssh-key";  
    public_key = '''\${file("~/ssh/id_ed25519.pub")}\''';  
  };  
}
```

Сечение 4

Въпроси?